

Functions and automated code analysis

Reading

- [PY4E chapter 4](#)

Introduction

So far, we have learned how to write Python code for doing things one after another in a specific sequence. For instance, we can construct a word like this:

```
In [1]: suffix = "ed"
        stem = input("Specify a verb stem: ")
        word = stem + suffix
        print("The resulting word is:", word)
```

Specify a verb stem: lik
The resulting word is: liked

Let's try it again with the verb "hike":

```
In [2]: suffix = "ed"
        stem = input("Specify a verb stem: ")
        word = stem + suffix
        print("The resulting word is:", word)
```

Specify a verb stem: hik
The resulting word is: hiked

We got the right answer (... partly because we picked the right verbs), but we had to write the same 4 lines of code again. This kind of redundancy is bad, but fortunately Python has a mechanism for eliminating it.

Control question: why is redundancy bad? Give three different reasons.

Redundancy is bad because:

1. It makes code harder to maintain.
2. It increases the chance of mistakes.
3. It makes programs longer and harder to understand.

Functions

You have already been briefly introduced to functions. How would you apply functions in the previous examples of past tense verbs?

There is no exact answer to this question, but the most reasonable option would be to create a function that does this:

- Take a single argument of type `str`, containing a verb stem
- Return a `str` containing the past tense form of the given verb

That is, if the function is called `make_past`, you should be able to plug it into our example in the following way:

```
In [3]: stem = input("Specify a verb stem: ")
        past = make_past(stem)
        print("The resulting word is:", past)
```

Specify a verb stem: lik

```
-----
NameError                                Traceback (most recent call last)
Cell In [3], line 2
      1 stem = input("Specify a verb stem: ")
----> 2 past = make_past(stem)
      3 print("The resulting word is:", past)

NameError: name 'make_past' is not defined
```

This code won't run just yet, before we actually write the `make_past` function. But given the **specification** above we can still write code **using** the function in advance. This is a common way of programming, and it allows people to work independently on the same project. Even if you are working alone, it is still very useful to think of functions in terms of their **specifications** or **interfaces**, that is, how one would use the function. This includes:

- Arguments: how many, which type(s), allowed values
- Return value: which type, allowed values
- Exceptions (we'll get to these later)

One of the most common mistakes by Python beginners is the following, which we illustrate by trying to write a `make_past` function:

```
In [4]: def make_past(stem):  
        suffix = "ed"  
        word = stem + suffix  
        print(word)
```

What is wrong here? Let us test. From now on, we specify the stems directly in the code for convenience.

```
In [5]: stem = "lik"  
        past = make_past(stem)  
        print("The resulting word is:", past)
```

liked

The resulting word is: None

You see the right answer being printed, but not in the way you expected. This is a rather serious error, because the function does not do what it should. Specifically, it is supposed to return the word as a `str`, but instead prints the word to screen and returns `None` (which is the default if there is no explicit `return` statement).

Because the function does not follow its specification, the code using it makes invalid assumptions. In this case the assumption is that `past` is a string containing a word, while in fact it has the value `None`. Let's fix the problem:

```
In [6]: def make_past(stem):  
        suffix = "ed"  
        word = stem + suffix  
        return word
```

```
In [7]: stem = "program"  
        past = make_past(stem)  
        print("The resulting word is:", past)
```

The resulting word is: programed

Much better!

Visualizing function calls

New programmers are often confused by function calls, because they involve passing values and data around under different names.

Using a tool like the [Python Visualizer](#) can sometimes help you understand what is actually going on, by going through the code step by step and demonstrating with arrows and boxes exactly what Python is keeping in memory for you and how it is referenced in your code. Try copy-pasting the following into the Python Visualizer:

```
In [8]: def make_past(stem):
        suffix = "ed"
        word = stem + suffix
        return word

suffix = make_past("lik")
```

This code is a bit confusing, because the variable name `suffix` is used twice, for two different purposes. While frowned upon, it is still perfectly valid Python code, because **local variables defined inside a function are internal to the function and not visible from outside the function**. Thus, there is no risk that Python confuses the two `suffix` variables (whether *humans* confuse them is a different story).

The distinction becomes very clear when using the [Python Visualizer](#). Press the "Visualize execution" button, then the "Next >" button repeatedly to follow the program step by step. Here is what happens:

Step	Action
1	Python reads and saves the function definition to memory, under the name <code>make_past</code>
2-3	Python calls the function <code>make_past</code> , assigning the value <code>"lik"</code> to the argument <code>stem</code>
4	Python assigns the value <code>"ed"</code> to the local variable <code>suffix</code>
5	Python concatenates the values of <code>stem</code> (<code>"lik"</code>) and <code>suffix</code> (<code>"ed"</code>), obtaining <code>"liked"</code> , which is assigned to the local variable <code>word</code>
6	Python returns the value of <code>word</code> (<code>"liked"</code>) from the function
7	Python assigns the return value of the function to the variable <code>suffix</code> . Note how the local variables in <code>make_past</code> fade away, to indicate that they are thrown away by Python as soon as the function returns

Functions are values

In Python, functions can be assigned to variables or passed as arguments, just as strings, integers, lists and other more common Python data types. For instance, this is perfectly valid code:

```
In [9]: def reduplicate(word):
        return word + word

def shorten(word):
    return word[:-1]

def print_word(word, transformation):
    transformed_word = transformation(word)
    print(transformed_word)
```

mypy

Purpose: Checks type errors in Python.

Python normally does not require you to specify data types, but mypy checks if you use the correct types.

Example:

```
def add(a: int, b: int):
    return a + b
```

Now, we can choose which transformation we want to pass to `print_word`, in order to print a word reduplicated, shortened, or changed in some other way that we may decide on later.

```
add("hello", 5)
```

```
In [10]: print_word("going", reduplicate)
```

mypy warns:

goinggoing

Expected int but got string

```
In [11]: print_word("going", shorten)
```

Think of it as a type checker.

goin

This is in fact a very powerful mechanism, which is used extensively in *functional programming* and occasionally in Python as well. You can use it to customize elements of how a function behaves, without changing the function itself.

Automated code analysis

Python is a fairly liberal language, in the sense that few things are mandatory compared to many other programming languages. Instead, an increasing amount of *voluntary* extensions have been added to Python and its ecosystem of external tools.

1. pylint

Purpose: Checks your Python code quality and style.

You have probably seen some of these tools already: the `pylint`, `mypy` and `pycodestyle` programs, which are integrated in the `micro editor on the server`. Whenever you save a Python program, these tools will go through your code and report any potential problems they detect to the editor, which displays them using `>>` symbols to the left of the affected code lines.

It looks for:

- Programming mistakes
- Bad coding practices
- Unused variables
- Naming problems
- Code style issues

Example:

```
x=10
print(y)
```

pylint may warn:

y is not defined

Type annotations

In order to help `mypy` do its job, catch some additional bugs, and make your code easier to follow, you can tell Python what data type variables *should* be. Consider the example from earlier:

```
In [12]: def make_past(stem):
          suffix = "ed"
          word = stem + suffix
          print(word)
```

pycodestyle

Purpose: Checks Python formatting style according to PEP 8 rules.

It checks things like:

Spaces
Line length
Indentation
Formatting

This does not do what we intended to do (i.e. *return* the word with the suffix added), but rather prints the word and returns `None`. But the tools will not catch this problem for you, because this is valid Python code which *does* something, and there is no way for the computer to know that you *intended* to do something different.

With type annotations, the code would look like this:

Example:

```
In [13]: def make_past(stem: str) -> str:
          suffix: str = "ed"
          word: str = stem + suffix
          print(word)
```

Bad:

x=10

Better:

x:=10

What you are telling Python is that the argument `stem` should be a string (`str`), and the same goes for the local variables `suffix` and `word`. In addition, by adding `-> str` after the parentheses in the function definition, you are saying that the function `make_past` is supposed to return a `str`. This last piece of information is crucial, because `mypy` can see that there is no `return` statement in your code that could return a `str`, and will report this.

pycodestyle reports style problems!

Think of it as a formatting checker.

Note that even though `mypy` discovers this discrepancy between what you wanted to write and what you actually wrote, this is still perfectly valid Python code and you can use it to print (but not return) a word:

```
In [14]: make_past("lik")
```

liked

How to work with automated checking tools

You should treat these tools like an imperfect spell checker, or perhaps a nagging parent. In the beginning it is difficult to know whether the problems reported are minor aesthetic issues or if they will crash your program. The tools are sometimes not very good at making this distinction, either. To be on the safe side, you could just try to fix everything. When you become more accustomed to the tools and to Python programming in general, you will learn which warnings you can safely ignore in case you are in a hurry.

That said, do not underestimate the "aesthetic" comments. Readability is an important issue in programming, and breaking conventions can lead to confusion and misunderstandings which can in turn lead to more serious mistakes.